Bradley S. Isenbek

AI systems architect and machine learning engineer: agent platforms, data architecture, and the hardware they run on.

brad@bradley.io · bradley.io · github.com/tinymachines · Grand Rapids, MI

Open to full-time roles in AI systems and data architecture. Remote, or on site in the Grand Rapids area.

SUMMARY

AI systems architect and machine learning engineer with more than fifteen years designing secure, large-scale data systems for government and enterprise clients, including a classified project for the United States Government and infrastructure processing billions of records. Expert in AI and ML pipelines, distributed systems, and secure data management.

Since February 2026: Campaign Brain, a production-ready platform of 84 self-hosted services on the SysForge fabric, with a journaled state kernel for agents, a declarative execution engine and planner, a service mesh, and an intelligence crawler; MyFinalWishes, a digital estate-planning product shipped through SysForge; a transistor-level 6502 and a switch-level NES at tinymachines.ai; and TerraPulse, an open climate and geophysical data platform. Daily co-development with Claude and other frontier models, with every working session and its cost on the public record.

EXPERIENCE

Founder and AI Systems Architect, SysForge.ai — 2024 to present

AI systems and development firm. Since 2026 the work has centred on Campaign Brain, a production-ready platform built on the SysForge fabric, on products shipped through it, and on the self-hosted infrastructure under both.

CAMPAIGN BRAIN, THE PLATFORM

- Built Campaign Brain, a production-ready platform on the SysForge fabric: 84 self-hosted Python services exposing about 3,390 documented endpoints, where every read endpoint is also an MCP tool for connected models. More than 20,000 commits of his own across more than 140 repositories since February 2026, pair-programmed with AI, not counting automated chores and merges.
- Designed a journaled state kernel for agents: five classical primitives (frontier, seen set, memo, plan DAG, disjoint groups) behind one loop, where the search strategy is configuration, not code. Every change goes to an append-only journal, so a crashed run replays to its exact next step and a supervisor can read an agent's state without asking it. It ships with a static lint whose three rules were each found first in production graphs.
- Built a declarative execution engine that merged three in-house workflow engines into one. Graphs are written as YAML and run once, run durably (a run can sit at an approval gate for days and resume exactly once by replay), or run as long-lived reconcile services.
- Built a planner on that engine. Two plain sentences go in. A typed judge picks a catalog template or a model writes a new graph, and a check against the live service catalog rejects any call to an operation that does not exist. Nothing runs until a person clicks Run, and anything that leaves the building waits at an approval gate.
- Built the service mesh the platform runs on. Each service announces a card describing itself, peers find each other by broadcast and seed lists, and membership expires by TTL, so a dead node simply ages out. Services are called by name through one gateway.
- Built the platform's intelligence crawler: an async jobs API that runs YAML pipelines (crawl, archive retrieval, screenshots, transcripts, vector search), running on a self-built cluster of crawl workers. Its largest measured run processed about 950 GB in nine days.

DISASTER RECOVERY

- Rehearsed a from-nothing rebuild of the platform on a second host: fresh secrets, empty schemas for 63 managed databases and 47 storage buckets, one tenant and one admin, then every service walked in dependency order. 67 of 93 units started clean, and the other 26 became tickets grouped by cause.
- Turned that rehearsal into the platform's first fresh-host runbook: host preparation, secrets bootstrap, the order of the core services, and a verification checklist, with the layout assumptions that had lived only in people's heads written down.
- Designed a whole-server encrypted offsite backup. It splits each disk into chunks that fit a nightly window, caps bandwidth, refuses to run if key material would leave the building, and uses credentials that cannot erase backup history. Seeding is in progress.
- Restored a 235 MB database from its nightly backup into a throwaway catalog in 33 seconds and checked the counts against live with no mismatches, then wrote the result up as a reusable restore runbook.
- Designed snapshot-based ZFS replication for an embedded database and its file blobs, with a failover guard that refuses to promote a standby that is behind on replication.

MYFINALWISHES, SHIPPED THROUGH SYSFORGE

- Built MyFinalWishes, a digital estate-planning platform, as its only engineer: rebuilt from an earlier prototype into a single TypeScript application (TanStack Start, React 19, SQLite with Drizzle), 459 commits of his own and 68 schema migrations from July to September 2026.
- Designed per-user envelope encryption for identity data and stored files: AES-256-GCM under a per-user key wrapped by a master key held outside the database, each sealed value bound to its user, table and column, and files sealed in 64 KiB chunks so ranged reads stay authenticated.

- Took it to production with a replicated standby that can take over, nightly verified off-site backups, and a deep health check that proves sign-in, decryption and mail delivery rather than pinging. About 1,500 automated tests, unit and Playwright end to end, gate every change in CI.

INFRASTRUCTURE

- Run a multi-domain web estate of more than 50 hostnames on owned hardware, with nginx, systemd, certbot and self-hosted BIND DNS, version-controlled units and encrypted secrets. No cloud host.

`MyFinalWishes: mywishes.com`

`Python · FastAPI · TypeScript · Claude API · MCP · SQLite · ZFS · WireGuard · OpenWrt · systemd · nginx`

Research and open source, tinymachines.ai, TerraPulse, tinychase

`2026 to present`

Chips simulated from die photographs, an open climate and geophysical data platform, and a cyclone model that runs in the browser. Built in public, on owned hardware.

TINYMACHINES.AI

- Designed and shipped tinymachines.ai in six weeks (763 commits, August 22 to October 2, 2026): one self-hosted site over four projects on Next.js 16, FastAPI and Rust/WebAssembly, with an API spoken as both REST and MCP and documentation in English and Japanese. Its headline figures are read from data files written only by scripts, 42 Playwright specs check the live site, and a deploy is refused when a served WebAssembly bundle does not match its recorded source commit.
- Built a transistor-level (switch-level) MOS 6502 simulator in Rust and WebAssembly on halfphi, an MIT-licensed library for switch-level simulation of chips traced from die photographs: 1,725 nodes, 3,510 switches and no behavioural model, matching the visual6502 reference bit for bit on every node across a 3,000 half-cycle trace. It has a WebGL2 die renderer, an HTTP API whose simulation calls are stateless, and an MCP endpoint.
- Designed an engine ladder in which each faster engine is derived from the switch-level chip and held to it at the pins: a micro engine at 39.0 M half-cycles a second (about 1,465 times the switch-level engine) and a GPU kernel stepping 128,000 machines at once, with all 289 recorded pin traces replaying exactly and each comparison shown to fail under a deliberate mutation.
- Extended the method to the NES: switch-level Ricoh 2A03 and 2C02 chips, an NTSC signal and CRT model, and a console with seven cartridge board types that passes blargg's CPU instruction and timing, sprite-hit and APU test ROMs, across eight public repositories and more than 830 commits in seven weeks.
- Built the bench that holds the model to the real console under identical inputs: an inline bridge between an unmodified NES and an original pad (a shift register the console clocks, a microcontroller counting its latch and clock pulses in hardware) and a Raspberry Pi head that drives reset, power and the oscilloscope from scripts.

TERRAPULSE, ARCHITECT AND BUILDER

- Architected and built TerraPulse, a measured-data climate and geophysical platform: 80 published per-phenomenon indexes holding 14.2M records (8.9M measured events and series plus a 5.3M-entry EPA facility directory), citing 183 upstream datasets from USGS, NOAA, NASA, EPA, USDA and others, on PostgreSQL/PostGIS, DuckDB and Parquet, self-hosted with systemd and nginx.
- Designed and shipped its public Model Context Protocol server: 11 read-only tools over the indexes, research workspaces and a 1,353-node knowledge graph, with a machine-generated spec whose examples run against live data, connected to claude.ai as a custom connector.

TINYCHASE, ENGINEERED

- Engineered tinychase.com in nine days (128 commits, AI pair-programmed, building on the TerraPulse data lake and model): an installable PWA where DuckDB-WASM range-reads static Parquet and a tinygrad-exported two-layer cyclone transformer runs in the browser on WebGPU, checked against the Python reference on a test storm (0.0000064 kt difference).

BRADLEY.IO

- Run bradley.io on owned hardware with no cloud host: a Next.js 16 site with live dashboards fed by real instruments, a public record of every AI working session and what it cost, and a staged deploy with automatic rollback used for more than 400 releases since February 2026.

`tinymachines.ai` · `tinymachines.ai/6502` · `terrapulse.info` · `tinychase.com` · `github.com/tinymachines`

`Rust · WebAssembly · WebGL2 · WebGPU · tinygrad · Next.js · FastAPI · PostgreSQL/PostGIS · DuckDB · Parquet · MCP`

Architect and Developer, VictoryText, LLC

`Grand Rapids, MI · 2022 to present`

Enterprise-grade, high-volume messaging platform with strict uptime and compliance requirements.

- Architected a high-availability messaging platform handling millions of messages at 99.9% uptime, using AI for anomaly detection and delivery optimization.
- Developed the RESTful API layer in FastAPI, integrating multiple carriers (Twilio, Bandwidth, Kaleyra, Telnyx) with automated failover.
- Engineered DynamoDB operations for atomic queue management, ensuring message delivery guarantees.
- Built a real-time analytics backend with AI-driven pattern recognition, aggregating carrier statistics through AWS QuickSight.

`Python · FastAPI · DynamoDB · Lambda · CloudWatch · QuickSight`

Architect and Developer, ConservativeConnector, LLC

`Grand Rapids, MI · 2018 to 2022`

Large-scale email data management systems processing billions of messages and more than 100 million contact records.

- Designed high-volume data infrastructure on AWS, with ML-driven data quality scoring and deduplication.
- Integrated and normalized 4.9 billion data points for ML model training, relationship discovery, and predictive analytics.
- Built multi-channel ingestion pipelines with ETL processes shipping to Snowflake, incorporating automated anomaly detection.
- Ensured GDPR and CAN-SPAM compliance across all data processing through automated policy enforcement.

`Python · AWS · Snowflake · PostgreSQL · ML pipelines · ETL`

Senior Architect, TransUnion, TruLookup Division — Boca Raton, FL · 2014 to 2018

Investigative intelligence platform providing insight on people, assets, and relationships from more than 10,000 data sources. Served law enforcement, government agencies, and legal clients.

- Lead developer on a classified project with the United States Government, covering secure system architecture and federal compliance.
- Architected distributed infrastructure processing billions of records and millions of queries a day at better than 99.9% uptime.
- Built machine learning models for entity resolution, relationship mapping, and data segmentation with the data science team.
- Implemented ML-driven search algorithms connecting disparate data sources to reveal hidden relationships between individuals, businesses, assets, and locations.
- Led a team of five developers in search algorithm optimization, performance tuning, and ML model integration.

C# · .NET · Apache Solr · Elasticsearch · SQL Server · Distributed caching

EARLIER CAREER, 1997 TO 2014

2012 to 2014 **Senior Architect**, nextSource / PeopleTicker, Manhattan, NY. Talent acquisition automation. PeopleTicker was a start-up spun out of nextSource.

2011 to 2012 **Application Developer**, nextSource, Manhattan, NY

2009 to 2011 **Senior Developer**, Linx Communications, Inc., Hauppauge, NY. Digital marketing.

2008 **Software Development and System Design**, J. Lack Consulting, East Hampton, NY

2008 **Senior Developer**, EnablePay, Inc., Mineola, NY. Credit card processing start-up.

2007 to 2008 **Senior Developer**, AmeriCorp, Inc., Syosset, NY. Debt consolidation.

2005 to 2007 **Senior Developer**, Evo Merchant Services, Inc., Melville, NY. Credit card processor.

2004 to 2005 **Senior Developer**, Direct Insite, Inc., Hauppauge, NY. Invoicing services for IBM.

2002 to 2004 **Senior Developer**, CCSI, Hauppauge, NY. Technology contractor.

1997 to 2002 **Chief Technology Officer and co-founder**, PipeLive, LLC, Melville, NY. Real-time chat and collaboration software, sold as a service.

Awards: PC Magazine Winner's Circle (PipeLive, LLC, 1997 to 2002); TMC Labs Innovation Award (PipeLive, LLC, 1997 to 2002).

GOVERNMENT AND CLASSIFIED WORK

- Lead developer on a classified project with the United States Government (TransUnion TruLookup, 2014 to 2018).
- Extensive experience in secure environments handling sensitive investigative and personal data for law enforcement and federal agencies.
- Working knowledge of data protection protocols, federal compliance frameworks, FISMA, and security practice.

OPEN SOURCE AND PROJECTS

bradley.io AI pilot: A public record of AI-augmented development: every working session, the models used, and what it cost.

Sovereign: An assembly-like agentic programming language designed for self-improving AI systems.

Gene Pool: A curated archive and analysis of AI coding agent system prompts across frontier models.

ARI: An AWS resource inventory tool.

ESP32 AI Edge: Edge computing and embedded inference on ESP32 mesh networks.

Rust for Haters: Systems programming deep-dives.

EXPERTISE

AI and machine learning	Claude and the Anthropic API · Model Context Protocol (MCP) · Agent state and orchestration · OpenAI API · Agentic frameworks · Prompt engineering · RAG pipelines · Vector databases · Hugging Face · Entity resolution · Relationship mapping · Recommendation engines · Statistical modeling · scikit-learn · Weka
Languages	Python · Bash · SQL · C# and .NET · Rust · C and C++ · Java · JavaScript and TypeScript
Cloud and infrastructure	AWS · Linux administration · Docker · Distributed systems · High-availability design · Load balancing · Disaster recovery · WireGuard and OpenWrt · ZFS · nginx, systemd and BIND
Data and search	Snowflake · PostgreSQL · MySQL · SQL Server · DynamoDB · Redis · Elasticsearch · Apache Solr · ETL pipelines · Hadoop and Hive · Vector embeddings · SQLite · DuckDB and Parquet · PostGIS
Development	FastAPI · REST APIs · Microservices · Git · CI/CD · Message queues · Real-time processing

EDUCATION

Advanced Object-Oriented Programming, Columbia University · **Computer Science coursework**, Plymouth State College · **Physics Engineering**, University of Buffalo